

File Encryption Project:

Description:

The goal of our project is to simulate an attack. The attack consists of encrypting/decrypting all of the users files recursively with a key. The encrypting/decrypting process would be multithreaded to increase the speed of the attack. For encrypting/decrypting we want to use the Advanced Encryption Standard (AES) algorithm specifically, the AES-128-CBC from the OpenSSL software library.

Steps sequence:

1. The victim opens attachment with script
2. The script encrypts users files with a generated key
3. The key is sent to the attacker
4. A text file is generated with instructions to get the data back
5. The victim follows the instructions
6. The attacker sends the key to decrypt all their files.

Requirements:

- C language
- Linux Os
- OpenSSL software library

Questions:

- **More sophisticated network protocol for key exchange:**
We would want to use a simple TCP connection. Here we would learn how the operating system handles system calls like `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()` and/or `close()`. In the "real world" this would not be the way it's done, since you can easily trace where the key was sent, but since this is a Uni project we think the learning about masking the key exchange would not add much to learning about OS. If necessary we can think about switching protocols to something like HTTPS, but the core concept of having a partner program for receiving the key running on the attacker's machine would stay the same.
- **Clarify what you implement yourselves vs what OpenSSL provides:**
The only thing provided by OpenSSL is the AES algorithm to encrypt/decrypt the files. The rest would be implemented by us.
- **Monitor component:**
We suggest two different options for the monitor component.

Option 1: The Monitor as a Defender (Prevention)

The goal is to build a standalone security program that monitors the file system and detects/kills the files encryption process, for that we could use fanotify.

Option 2: The Monitor as an Accomplice (Continuous Attack)

The goal is to make the attack more persistent by having a program that constantly runs in the background after the encryption. It monitors the whole files systems for newly added files which then get encrypted as well. To implement this we would use inotify, inotify_add_watch.

- **How do you handle permissions?**

During traversal, we attempt to access files and directories, if access is denied, we skip that entry and continue traversing.

- **How will you traverse the filesystem?**

The filesystem is traversed recursively.

For handling files we will rely on: openat, read, close and stat. For traversing directories, we will use getdents64 with openat and stat. We will open directories using openat, read their entries using getdents64 and use stat to determine if the entry is a file or a directory and its permissions.